

Scientific Computing Exercise Set 1

Abinash Satapathy (id) & Konstantinos Benjamin Vigos (16393929) & Iga Pręgoska (id)

Repository: <https://github.com/BenVigos/Scientific-Computing>

I. INTRODUCTION

NUMERICAL computations provide an essential framework for solving complex partial differential equations that describe various physical systems. Our investigation begins with the numerical solution of a one-dimensional wave equation, which might describe the vibrations of a uniform string, sound waves in gases or liquids, and the transport of voltage along a lossless transmission line. The continuous solution, $\Psi(x, t)$, represents the vibration amplitude expressed as a function of position and time. To make the problem fully posed, specific boundary conditions are applied on the spatial domain, alongside initial position and velocity distributions. For a string of length $L = 1$, the boundaries are fixed such that $\Psi(x = 0, t) = 0$ and $\Psi(x = L, t) = 0$. By introducing a uniform discretization of the spatial and temporal domains, we approximate the partial differential equation using finite difference expressions. The time development of the system is modeled using time-stepping algorithms, including the Leapfrog method. This explicit, second-order symplectic integrator preserves the underlying phase-space structure of Hamiltonian systems. As a result, it keeps the energy error bounded over long simulation times, making it highly effective for conservative and oscillatory systems.

The second phase of this study shifts focus to mass transport, specifically modeling the two-dimensional time dependent diffusion equation. The governing partial differential equation is defined as:

$$\frac{\partial c}{\partial t} = D \nabla^2 c \quad (1)$$

where $c(x, y; t)$ represents the concentration as a function of coordinates and time, and D is the diffusion constant. The computational domain is restricted to a square where $0 \leq x, y \leq 1$. The system is subjected to specific boundary constraints: the top of the domain is fixed at $c(x, y = 1; t) = 1$, and the bottom is fixed at $c(x, y = 0; t) = 0$. Additionally, periodic boundary conditions are applied in the x-direction, meaning $c(x = 0, y; t) = c(x = 1, y; t)$. We discretize the spatial and temporal domains into intervals of length δx and time steps δt , defining the concentration at discrete points as $c(i\delta x, j\delta x; k\delta t) \equiv c_{i,j}^k$. Utilizing a standard

5-point stencil, we apply the following explicit finite difference scheme:

$$c_{i,j}^{k+1} = c_{i,j}^k + \frac{\delta t D}{\delta x^2} (c_{i+1,j}^k + c_{i-1,j}^k + c_{i,j+1}^k + c_{i,j-1}^k - 4c_{i,j}^k) \quad (2)$$

This calculation scheme relies strictly on local information for updates, making it well-suited for parallel computations. However, the time step is limited by the stability condition $\frac{4\delta t D}{\delta x^2} \leq 1$.

When the transient behavior of the concentration profile is not the primary focus, it is more efficient to directly study the steady state by solving the time independent diffusion equation. In this scenario, the system reduces to the Laplace equation:

$$\nabla^2 c = 0 \quad (3)$$

Applying the same spatial discretization and evaluating the second-order derivatives yields a simplified set of finite difference equations:

$$\frac{1}{4} (c_{i+1,j} + c_{i-1,j} + c_{i,j+1} + c_{i,j-1}) = c_{i,j} \quad (4)$$

To solve this system efficiently and minimize memory demands compared to direct solvers, iterative methods are utilized. The most fundamental approach explored is the Jacobi iteration, defined mathematically as:

$$c_{i,j}^{k+1} = \frac{1}{4} (c_{i+1,j}^k + c_{i-1,j}^k + c_{i,j+1}^k + c_{i,j-1}^k) \quad (5)$$

Unlike time dependent simulations, iterative methods require a global stopping condition to determine convergence. For this study, the solution is considered converged when the maximum change between iterations is smaller than a specified tolerance ϵ :

$$\delta \equiv \max_{i,j} |c_{i,j}^{k+1} - c_{i,j}^k| < \epsilon \quad (6)$$

Implementing the Jacobi iteration requires separate matrices for $c_{i,j}^k$ and $c_{i,j}^{k+1}$ to prevent overwriting the necessary values. This foundational setup provides the necessary framework for exploring more advanced and faster-converging iterative algorithms, such as the Gauss-Seidel iteration and Successive Over Relaxation (SOR). Gauss-Seidel method provides marginal improvement over the Jacobi scheme by using updated values in each iteration, removing the need for separate matrices.

SOR further generalises Gauss-Seidel by introducing a relaxation parameter ω :

$$c_{i,j}^{k+1} = \frac{\omega}{4}(c_{i+1,j}^k + c_{i-1,j}^k + c_{i,j+1}^k + c_{i,j-1}^k) + (1-\omega)c_{i,j}^k. \quad (7)$$

II. METHODOLOGY

A. Vibrating String

In order to explore the behaviour of a vibrating string the one-dimensional wave equation is implemented using a second order accurate finite difference method.

The approximation of the 2nd derivative is derived by summing the Taylor expansions of $f(x+h)$ and $f(x-h)$ around x , and solving for the second derivative, as seen in eq. (8)

$$f^{(2)}(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} + \mathcal{O}(h^2) \quad (8)$$

Following this process for both the spatial and time derivatives of the wave equation, and writing the results in vector form, we get:

$$\vec{\psi}_j^{i+1} = 2\vec{\psi}_j^n - \vec{\psi}_j^{n-1} + \left(\frac{c\Delta t}{\Delta x}\right)^2 (\mathbf{A}\vec{\psi}^n + \vec{b}), \quad (9)$$

Applied on the interior points, $j = 2, \dots, n-1$

Where

$$\mathbf{A} = \frac{1}{\Delta x^2} \begin{bmatrix} -2 & 1 & 0 & \cdots & 0 \\ 1 & -2 & 1 & \ddots & \vdots \\ 0 & 1 & -2 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & 1 \\ 0 & \cdots & 0 & 1 & -2 \end{bmatrix} \in \mathbb{R}^{(N-2) \times (N-2)}$$

and

$$\vec{b} = \frac{1}{\Delta x^2} \begin{bmatrix} g_0 \\ 0 \\ \vdots \\ 0 \\ g_N \end{bmatrix} \in \mathbb{R}^{N-2}$$

where N is the number of grid points, and g_0 , and g_N , are the Dirichlet boundary conditions of the left and right boundary respectively.

For all simulations a timestep of $\Delta t = 0.01$, and wave velocity of $c = 1$ is used. In order to ensure stability the CFL criterion, as seen in eq. (10) requires that $\Delta x \geq c\Delta t$

$$c \frac{\Delta t}{\Delta x} \leq 1 \quad (10)$$

B. Time Dependent Diffusion Equation

To investigate the two-dimensional time-dependent diffusion equation, a numerical simulation was developed using an explicit finite difference scheme. The numerical results were subsequently validated against an exact analytical solution to ensure the fidelity of the computational model.

C. Domain Discretization and Stability Criteria

The governing time-dependent diffusion equation is given by equation ??

The spatial domain was discretized into $N = 50$ intervals along both axes, yielding a uniform grid spacing of $\delta x = \delta y = 1/N$. Time was discretized into steps of δt . Applying a standard explicit finite difference approximation to Equation ?? yields the numerical update scheme as in equation 5.

To guarantee numerical stability for this explicit five-point stencil, the time step δt must strictly satisfy the von Neumann stability condition:

$$\frac{4\delta t D}{\delta x^2} \leq 1 \quad (11)$$

In the computational implementation, δt was dynamically calculated using a safety factor of 0.2 to ensure robust adherence to this theoretical limit.

D. Initial and Boundary Conditions

The simulation was initialized with a zero-concentration state across the domain, $c(x, y; t = 0) = 0$. The boundary conditions dictated by the physical system were rigorously enforced at each time step:

- **Dirichlet Boundaries (y-axis):** Fixed concentrations were maintained at the top and bottom of the domain, such that $c(x, y = 1; t) = 1$ and $c(x, y = 0; t) = 0$ respectively. Computationally, the first and last rows of the concentration matrix were explicitly overwritten with these fixed values after every iteration.
- **Periodic Boundaries (x-axis):** The domain exhibits periodicity in the x -direction, formulated as $c(x = 0, y; t) = c(x = 1, y; t)$. To implement this without requiring ghost nodes, modulo arithmetic was utilized to map the left neighbor of the $i = 0$ boundary to $i = N - 1$, and the right neighbor of $i = N$ to $i = 1$. This seamlessly wraps the grid array, simulating an infinite lateral expanse.

E. Data Logging Protocol

To facilitate temporal analysis without exceeding memory constraints, a periodic data logging protocol was implemented. Every 100 iterations, the numerical array was sampled, and the 1D concentration profile along the y -axis (extracted from the geometric center of the x -domain due to lateral symmetry) was appended to a CSV file. This lightweight tracking allowed for efficient post-processing and graphical rendering.

F. Analytical Verification

To verify the accuracy of the numerical solver, the simulated profiles were compared against the exact analytical solution. Because the boundary conditions and initial state are uniform across x , the true solution is invariant with respect to the x -coordinate, reducing the problem to a 1D functional dependence on y and t .

The analytical solution is expressed as an infinite series of complementary error functions:

$$c(y, t) = \sum_{i=0}^{\infty} \left[\operatorname{erfc} \left(\frac{1-y+2i}{2\sqrt{Dt}} \right) - \operatorname{erfc} \left(\frac{1+y+2i}{2\sqrt{Dt}} \right) \right] \quad (12)$$

Note that the spatial variable x in the source documentation has been adapted to y here to correctly align with the orientation of our boundary conditions.

The Python implementation computed this analytical solution truncated to the first 20 terms, as the series converges extremely rapidly. The numerical solution data was successfully captured and plotted against the analytical curves at discrete time intervals $t \in \{0.001, 0.01, 0.1, 1.0\}$. The convergence of the discrete scatter points onto the continuous analytical curves served as primary validation of the explicit finite difference methodology.

This is to cite the plot 1

G. Iterative Schemes

The Jacobi, Gauss-Seidel and SOR schemes (see I) were implemented on a $N \times N$ grid with uniform spacing of $\delta x = \delta y = 1/N$. Vertical and horizontal boundary conditions as described in section II-D were used. The tolerance for a stopping condition (see equation 6) was set to 10^{-5} across all experiments, and the iteration count k was defined as the number of full traversals of the interior grid points required to reach that tolerance.

For validation, each scheme was compared against an analytical solution [eq here], and their respective MSEs were plotted point-wise. To further investigate the properties of each scheme, a convergence plot was also created.

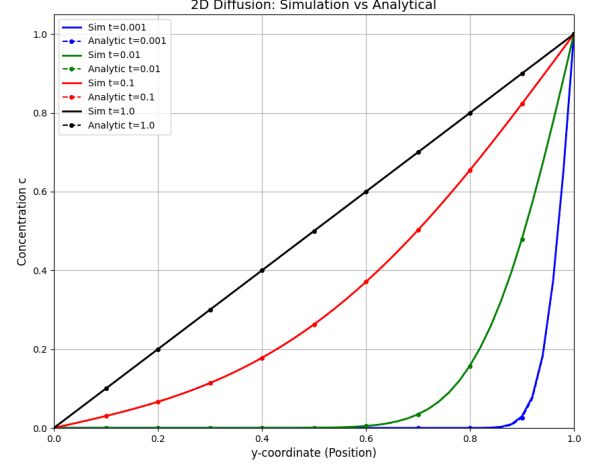


Fig. 1: Comparison of Simulation vs Analytical solutions for 2D diffusion

1) *Relaxation Parameter Dependence:* The optimal relaxation parameter ω_{opt} was defined as minimising the value of k needed for SOR convergence for a given problem. To determine the ω_{opt} an initial sweep was performed over $\omega \in [0.5, 2.5]$ with step size $\Delta\omega = 0.05$. Subsequently, a more refined sweep ($\Delta = 0.01$) was performed around the vicinity of the ω resulting in the minimal iteration count.

H. Internal Objects in a Grid

Internal objects were encoded using a boolean mask over a computational grid. Two types of objects were considered: sinks and insulators. For sinks, grid points belonging to the object mask were set to zero at each iteration. Insulators were implemented by enforcing a Neumann boundary condition ($\frac{\partial T}{\partial n}$) to equal zero at the grid points encoding edges of the insulator.

1) *Influence on the SOR Scheme:* In order to investigate the presence of objects on the SOR scheme, iteration counts were measured as a function of placement and size and final concentration fields were stored for comparison.

III. RESULTS

A. Vibrating string

First a grid convergence study is performed in order to determine a grid size that gives accurate results. This is done by varying the number of grid points logarithmically between 10 and 1000 - which is the maximum allowed based on the stability condition - and comparing the mean squared error of the solution after

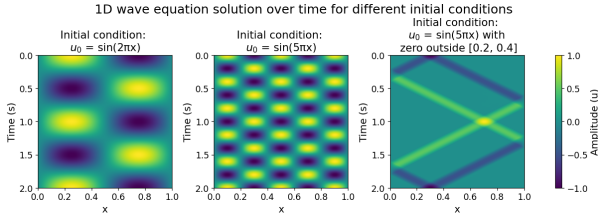


Fig. 2: Caption

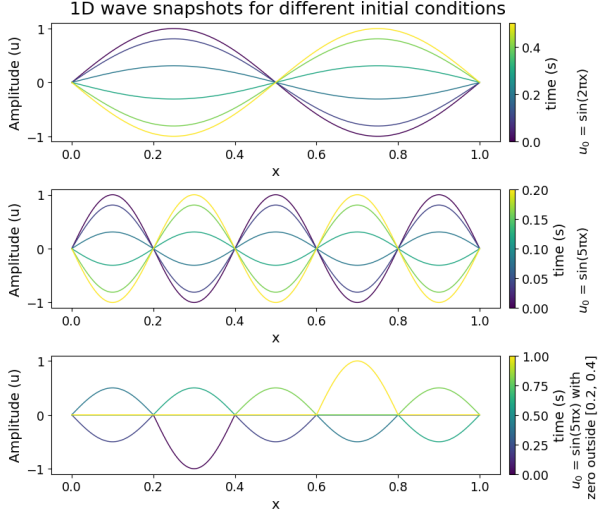


Fig. 3: Caption

one period, for initial condition 3 as seen in table I, to that with $N = 1000$

B. Comparison of Iterative Schemes

The Jacobi, Gauss-Seidel and SOR schemes were first compared on the base domain without the presence of internal objects, with a grid size of $N = 50$, and a tolerance of $\epsilon = 10^{-5}$.

To verify correctness, the numerical steady-state solution obtained from each method was compared against the analytical solution [ref]. The pointwise mean squared error (MSE) distribution is shown in Figure 5. All schemes eventually converged within a specified tolerance, with Jacobi exhibiting the largest peak error, reaching approximately 9.47×10^{-5} . Gauss-Seidel reduced the peak error to 2.36×10^{-5} , while SOR ($\omega = 1.8$) decreased it further to 2.68×10^{-7} . In all cases, the error is symmetric with respect to the center of the domain:

TABLE I: Initial conditions used for wave equation

number	Initial condition
1	$\psi(x, t = 0) = \sin(2\pi x)$
2	$\psi(x, t = 0) = \sin(5\pi x)$
3	$\psi(x, t = 0) = \sin(2\pi x)$ if $1/5 \leq x \leq 2/5$ else $\psi = 0$

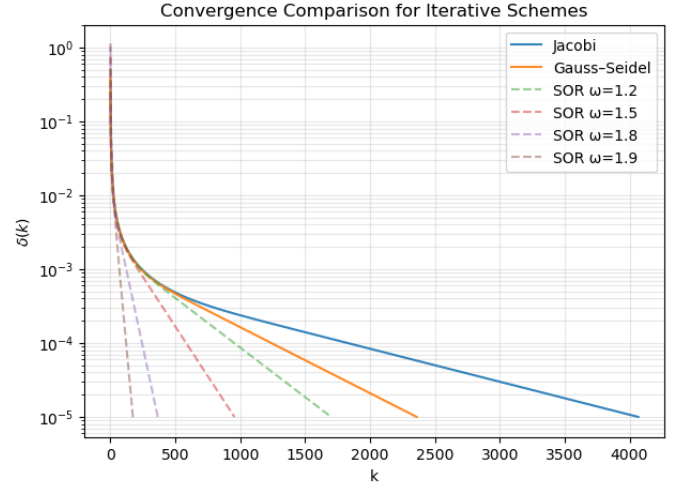


Fig. 4: Enter Caption

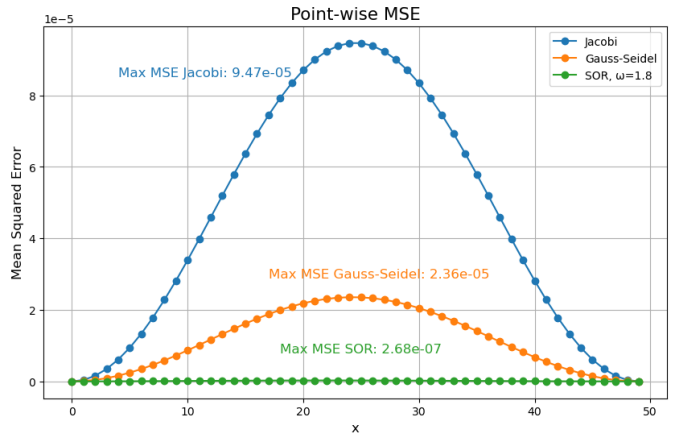


Fig. 5: Enter Caption

the error is minimal at the boundaries and maximal at the interior region.

Figure 4 shows the decay of the convergence measure δ as a function of iteration count. The Jacobi method showed the slowest convergence for the given problem ($k = 4068$), while Gauss-Seidel converged almost two times faster, with $k = 2361$. SOR exhibited a marked decrease in iterations needed with increasing relaxation parameter ($\omega \in 1.2, 1.5, 1.8, 1.9$), only requiring $k = 366$ iterations for the highest ω , making it over 11 times faster than the Jacobi scheme.

C. Optimal Relaxation Parameter

The dependence of the iteration count k on the relaxation parameter ω was investigated for grid sizes of $N = 20, 30, 50, 80, 100$. For each N , ω_{opt} was found, as shown in Figure

Due to the discrete sampling of ω with resolution $\Delta\omega = 0.01$ in the refined search, the found ω_{opt} has an uncertainty of approximately ± 0.005

D. Influence of Internal Objects

The presence of internal objects significantly affected convergence behaviour.

IV. DISCUSSION

V. CONCLUSION

APPENDIX

Extra components